系統架構文件（SA）

> 【版本說明】 本文件為 V8 系統之 旗艦版 **C# .NET 8** 商用級 MVP 開發時程專屬之客製化規格書。 此版本堅守雙向斷線容錯與基本資安底線，但依據時程嚴格刪減了非必要的自動化服務或進階報表機制。

集中式後台 + 單機本地後台整合版 (C# .NET 8)

屬性	內容
版本	v8.1
日期	2026 年 3 月 2 日
適用系統	遊戲平台管理系統
文件類型	系統架構

目錄

1. 文件概述
2. 系統架構概述
3. 集中式後台架構
4. 單機本地後台架構
5. 資料同步與對帳機制
6. 安全性架構
7. 效能設計
8. 部署架構
9. 附錄

---

1. 文件概述

本文件詳細闡述遊戲平台管理系統的整體系統架構，包含集中式後台與單機本地後台兩個層次的架構設計、資料流、通訊機制等內容。

2. 系統架構概述

2.1 中央集權架構

遊戲平台採用**中央集權架構**設計，所有單機本地後台必須先向集中式註冊才能啟用。

- **集中式後台**：負責全局資料監控、全平台交易統計、系統設定管理、**機台註冊與認證**。
- **單機本地後台**：受集中式管理的單機系統，負責本地機台、本地遊戲、本地交易結算。

> **核心設計原則**： - **單機本地後台**：必須連線方可啟動遊戲（保持連線方可進行交易，離線禁止開始新遊戲）。 - **匿名客端**：不留存外部個人資料，所有交易與餘額直接掛載於 **機台 (Machine)**。

2.1.1 運作模式

模式	說明	遊戲狀態
已註冊連線	正常向集中式註冊，網路暢通	✅ 可開始遊戲
已註冊離線	曾註冊但網路中斷	❌ 無法開始新遊戲
未註冊	未向集中式註冊	❌ 無法啟用

2.2 系統邊界定義

集中式後台邊界

邊界元素	說明
管理範圍	所有單機本地後台實例、全平台遊戲、全平台交易
資料角色	全域資料彙總、**機台認證權威**
可用性要求	99.5% 正常運行時間
網路依賴	持續連線（核心依賴）
職責	全域監控、帳務最終核對、**機台註冊認證**

單機本地後台邊界

邊界元素	說明
管理範圍	單一實例（單一機台）、本機遊戲、本地交易
資料角色	本地緩衝（需連線集中式才能運作）

邊界元素	說明
可用性要求	離線獨立運作能力（網路恢復後同步）
網路依賴	可離線運作（網路恢復後與集中式同步）
職責	本地遊戲執行、本地交易紀錄、離線緩衝

> 重要：單機本地後台支援離線獨立運作，連線恢復後自動與集中式同步資料。

邊界協定

協定類型	內容
通訊協定	HTTPS + WebSocket
資料交換格式	JSON
認證方式	JWT Token（集中）、PIN Code（本地）
連線要求	可離線運作（網路恢復後同步）
註冊機制	首次啟用需向集中式註冊取得授權

2.3 系統組件

系統包含以下主要組件：

- 集中式後台前端（Blazor Server + Radzen Blazor + Bootstrap 5）
- 集中式後台後端（ASP.NET Core 8 Web API）
- 單機本地後台前端（Blazor Server + Radzen Blazor + Bootstrap 5）
- 單機本地後台後端（ASP.NET Core 8）
- 集中式資料庫（PostgreSQL 15+）
- 單機本地資料庫（SQLite）
- 遊戲引擎
- 機台端應用程式

2.4 系統上下文關係圖

```
┌──────────────────────────────────────────────────────────────┐
│                          玩家端                              │
│                    (Mobile App / Web)                        │
└──────────────────────────────────────────────────────────────┘
                              │
                              ▼

┌──────────────────────────────────────────────────────────────┐
│                     單機本地後台 (Local)                      │
│  ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐     │
│  │ 遊戲引擎 │  │ 交易處理 │  │ 同步引擎 │  │ 監控代理 │     │
```

```
│   └──────────┘   └──────────┘   └──────────┘   └──────────┘       │
│          │              │              │              │           │
│          └──────────────┴──────────────┴──────────────┘           │
│                              │                                     │
│                          ┌───┴───┐                                 │
│                          │ SQLite │                                │
│                          └───────┘                                 │
└────────────────────────────────────────────────────────────────────┘
                              │ HTTPS + SignalR
                              │ (定時 Heartbeat)
                              ▼
┌────────────────────────────────────────────────────────────────────┐
│                      集中式後台 (Central)                          │
│   ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐          │
│   │ API Gateway │ │ 認證服務 │  │ 機台管理 │  │ 錢包服務 │          │
│   └──────────┘  └──────────┘  └──────────┘  └──────────┘          │
│   ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐          │
│   │ 遊戲管理 │  │ OTA 服務 │  │ 監控服務 │  │ [保留]   │          │
│   └──────────┘  └──────────┘  └──────────┘  └──────────┘          │
│          │              │              │              │           │
│          └──────────────┴──────────────┴──────────────┘           │
│                              │                                     │
│                          ┌───┴──────┐                             │
│                          │ PostgreSQL │                            │
│                          └──────────┘                              │
└────────────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌────────────────────────────────────────────────────────────────────┐
│                      遊戲商 API (Providers)                        │
│          ┌──────────┐    ┌──────────┐    ┌──────────┐             │
│          │ Provider A│    │ Provider B│    │ Provider N│            │
│          └──────────┘    └──────────┘    └──────────┘             │
└────────────────────────────────────────────────────────────────────┘
```

---

3. 集中式後台架構

3.1 前端架構

集中式後台前端採用 **Blazor Server + Radzen Blazor + Bootstrap 5**。使用 Blazor Router 進行路由管理，Radzen Blazor 提供豐富的 UI 元件，Chart.js 用於資料視覺化，Blazor State 進行狀態管理。支援響應式設計，適配桌面、平板、手機等多種裝置。

3.2 後端架構

集中式後台後端採用 **ASP.NET Core 8** 框架。提供 RESTful API 介面，支援 SignalR 實時通訊。使用 PostgreSQL 資料庫進行資料持久化。

3.2.1 服務分層架構

```
┌──────────────────────────────────────────────────────────────┐
│                      Presentation Layer                      │
│                 (Blazor Server + Radzen)                     │
└──────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────────┐
│                       API Gateway Layer                      │
│           (Authentication, Rate Limiting, Routing)          │
└──────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────────┐
│                     Business Logic Layer                     │
│                                                              │
│   ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐    │
│   │ Machine  │  │  Wallet  │  │  Game    │  │  OTA     │ │  │
│   │ Service  │  │ Service  │  │ Service  │  │ Service  │  │
│   └──────────┘  └──────────┘  └──────────┘  └──────────┘  │
│                                                              │
│   ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐  │
│   │ Provider │  │  Agent   │  │   OTA    │  │  Sync    │  │
│   │ Service  │  │ Service  │  │ Service  │  │ Service  │  │
│   └──────────┘  └──────────┘  └──────────┘  └──────────┘  │
└──────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────────┐
│                      │
│          ( Data Access Layer                    Entity Framework Core
+ Repository)          │
└──────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────────┐
│                       Infrastructure                         │
│        (PostgreSQL, Redis, SignalR Hub, File)              │
└──────────────────────────────────────────────────────────────┘
```

3.2.2 核心服務說明

服務名稱	職責	主要 API
MachineService	機台生命週期管理、餘額操作	CRUD, Balance, Command
WalletService	機台錢包交易、餘額對帳	Deposit, Withdraw
GameService	遊戲庫管理、上下架	CRUD, Enable/Disable

服務名稱	職責	主要 API
ProviderService	遊戲商管理、API 整合	CRUD, Balance Query
AgentService	代理商樹狀結構管理	CRUD, Hierarchy
OTAService	版本發布、進度追蹤	Upload, Publish, Status
SyncService	資料同步、衝突解決	Upload, Download

3.3 核心模組

集中式後台包含以下最高規的核心模組：

1. **儀表板模組**：全平台的自訂 Widget 戰情室。
2. **機台心跳深度監測模組**：(旗艦功能) 監控 CPU/RAM 與網路延遲，提早預防當機。預設心跳間隔 30 秒，可配置 10-300 秒。
3. **機台標籤管理模組**：(旗艦功能) 支援為機台新增自定義標籤（門市、區域、機型、客群等），支援多維度篩選與營收分析。
4. **遊戲與版本管理模組**：涵蓋遊戲授權與跨站點維護。
5. **OTA 版本發布中心**：(旗艦功能) 全球遠端熱更新腳本上傳與無縫派發。
6. **錢包交易模組**：全域交易流水帳與圖表化分析（以機台為中心）。
7. **爭議對帳處理專區**：(旗艦功能) 處理斷線異常、人工介入核銷專區與會計除錯。
8. **串接遊戲管理模組**：多家遊戲供應商 API 整合平台。
9. **多人遊戲模組**：多人遊戲（百家樂、龍虎、骰寶）的集中式管理，牌桌狀態監控與牌局歷史查詢。
10. **遊戲測試模組**：(旗艦功能) 允許管理員進行虛擬下注以驗證遊戲邏輯，不扣除真實餘額，記錄測試報告供稽核。
11. **使用者權限模組**：完全動態自定義的 RBAC 角色系統 (如：審計員、純查帳員)，並負責配置供本地端登入用的 4-6 位數 PIN 碼。
12. **介面設定模組**：玩家端與客製化版面的全域配置管理。
13. **營收報表模組**：(旗艦功能) 提供多維度營收分析報表，支援以下篩選維度（且可多選）：
    - 門市/分店維度
    - 機台維度
    - 機台標籤維度（支援多標籤篩選）
    - 遊戲維度
    - 遊戲商/供應商維度 支援日/週/月/年報表類型，包含欄位：總營收、下注總額、派彩總額、遊戲局數、營收(RTP)。

3.4 監控中心架構 (Monitoring Center)

監控中心負責全平台機台與伺服器的健康度狀態，採用實時串流與時序資料庫結合架構：

1. **資料收集層 (Data Collection)**

    - **單機代理 (Local Agent)**：每台實體機台透過本地背景服務，即時收集 CPU 使用率、記憶體佔用、網路延遲、硬碟空間與應用程式例外日誌。
    - **心跳機制 (Heartbeat)**：透過 HTTPS/WebSocket 定期（預設 30 秒）發送負載遙測數據至集中式後台。
2. **處理與儲存層 (Processing & Storage)**
    - **即時數據接收**：由集中端 ASP.NET Core 後端的 `MonitorService` 輕量化接收並校驗遙測數據。
    - **時序儲存**：高速讀寫性能指標並持久化至 PostgreSQL，保留歷史效能曲線供回放查詢。
3. **告警與展示層 (Alerting & Visualization)**
    - **規則引擎**：設定多種維度閾值（如 CPU > 90% 持續 5 分鐘，或網路斷線 3 分鐘），觸發管理員即時告警 (Alert)。
    - **戰情室面板**：前端 Blazor 大盤，包含機台地圖、高風險設備列表、效能歷史折線圖等。

---

4. 單機本地後台架構

> **設計原則**：單機本地後台為雲端快取端點（唯讀），不直接修改資料，所有操作透過雲端 API 執行。

4.1 前端架構

單機本地後台前端採用與集中式後台相同的（**Blazor Server + Radzen Blazor + Bootstrap 5**），確保設計風格與使用體驗的一致性。支援本地化部署，可在機台上直接運行。

4.2 後端架構

單機本地後台後端採用 **ASP.NET Core 8** 框架。提供 RESTful API 介面，支援 SignalR 實時通訊。使用 SQLite 作為本地資料儲存，僅用於快取與本地日誌。

4.3 核心模組

單機本地後台包含以下核心模組：

1. **操作與儀表板模組**：結合開洗分與現場活動捷徑的快搜面板（資料來自雲端快取）。
2. **熱更新接收器模組**：(旗艦功能) 於背景默默接收中央 OTA 封包、MD5 驗證並自我重啟 (不干擾客人)。
3. **深層機台監控模組**：(旗艦功能) CPU、記憶體佔用實時圖表並回傳。
4. **遊戲配置模組**：顯示本地快取的遊戲列表（唯讀，啟用/停用由雲端控制）。

5. **交易管理與非同步佇列模組**：本地交易暫存佇列，連線時同步至雲端（本地不直接寫入）。
6. **系統設定模組**：本地基本設定（網路參數等），重要設定由雲端統一管理。
7. **同步與對帳模組**：檢視雲端同步狀態，無法手動修改雲端資料。
8. **自動對帳系統**：(旗艦強化) 嚴謹地分析本地單據與中央回傳的 Ack 是否完全一致。
9. **機台底層通訊設定**：網路與中心通訊參數、金鑰加密防綁架設定。

4.4 本地端限制

> **中央集權架構**：本地端無法直接修改資料，確保資料一致性。

功能	本地端權限	說明
遊戲管理	❌ 唯讀	遊戲列表由雲端統一管理
系統設定	⚠️ 部分	僅本地網路參數，重要設定由雲端控制
交易紀錄	⚠️ 佇列	本地暫存，連線時上傳雲端
機台註冊	❌	必須連線至雲端註冊才能啟用
遊戲開始	❌	必須連線至雲端才能開始新遊戲

---

5. 資料同步與對帳機制

5.1 連線驗證機制

> **核心設計**：單機本地後台**必須連線至集中式才能開始遊戲**。 **本地端性質**：本地端為雲端快取端點，資料由雲端統一管理，本地無法直接修改。

狀態	遊戲	交易	本地資料
已註冊連線	✅ 可開始	✅ 可進行	雲端同步
已註冊離線	❌ 禁止開始	⚠️ 僅顯示歷史	唯讀快取
未註冊	❌ 禁止啟用	❌ 無法使用	無法使用

5.2 同步流程

單機本地後台連線 → 驗證註冊狀態 → 上傳本地交易佇列 → 下載集中式快取更新 → 完成

> **同步原則**：本地端**僅上傳交易資料**，下載雲端快取（遊戲、組態設定等），不直接寫入資料表。

5.3 離線處理

當單機本地後台離線時：

- **禁止開始新遊戲**：離線期間無法開始新遊戲（保護帳務）
- **保留歷史資料**：本地緩存的交易紀錄可檢視
- **連線恢復**：重新連線後同步至集中式

⚠️ **核心原則**：離線期間**禁止開始新遊戲**，確保帳務一致性。

5.4 對帳機制

系統定期進行對帳檢查，驗證本地資料與集中式資料的一致性。對於不一致的項目，記錄並標記為待審核，由管理員手動處理。

5.5 遠端控制機制

集中式後台可對單機本地後台進行遠端控制（需本地在線）。

操作流程：

```
1. 選定單機本地後台 → 選擇目標機台
        ↓
2. 驗證在線狀態 → 確認可連線
        ↓
3. 執行遠端操作 → 開分/洗分/重啟/設定等
        ↓
4. 套用並回傳結果 → 本地執行並回報
        ↓
5. 記錄日誌 → 儲存至集中DB
```

適用範圍：

功能模組	是否支援遠端控制
機台管理（開分/洗分/重啟/配置）	✅
遊戲管理（列表/配置/測試）	✅
系統設定	✅

5.6 匿名機台同步策略

系統採用匿名機台模式，所有額度與機台實體綁定，不支援跨店玩家資料同步。

5.7 跨店單機遊戲整合架構

針對單機遊戲（如老虎機、捕魚機等），跨店接續遊玩的技術整合流程著重於「開洗分（Deposit/Withdraw）」與本地交易快取：

1. **機台認證與登入 (Machine Authentication)**
   - **機制**：玩家於店家 A 的單機設備中登入或刷入 RFID 卡，機台即時向集中式後台請求此帳號的身分與對應餘額。
2. **本地開分與遊玩 (Local Deposit & Play)**
   - **開分 (Deposit)**：機台發送開分請求至集中式後台，將雲端的 Wallet 點數轉移/鎖定為該機台的「本地分（Credits）」做為遊玩額度。
   - **離線容錯與運算**：遊玩過程的引擎運算與每次 Spin 在單機本地完成。交易紀錄寫入本地 SQLite 佇列，並非同步拋回集中式後台。
3. **洗分與離機 (Withdraw & Release)**
   - **機制**：玩家選擇退出或拔出實體卡，主動結束該機台的遊玩。
   - **資料流**：機台統整結算點數（Credits），呼叫 Wallet API （Withdraw 或 Settle） 將本地餘額轉換回雲端 Wallet，並清除本機台內的玩家連線授權狀態。
4. **跨站點數延續 (Cross-Site Resumption)**
   - **機制**：玩家前往店家 B 操作另一台單人機台。只要在店家 A 的洗分流程已確認完成，店家 B 的機台便能透過即時身分驗證，取得最新的 Wallet 餘額繼續開分遊玩。
   - **防禦機制**：若玩家在店家 A 異常斷線未成功洗分，其集中式錢包會處於「機台鎖定 (Locked by Machine)」狀態，確保無法在店家 B 重複拿點數開分，直至店家 A 的異常解除或交由客服遠端結算。

---

6. 安全性架構

6.1 認證與授權

系統採用帳號密碼認證，使用 JWT Token 進行會話管理。全面實施基於角色的動態存取控制（Dynamic RBAC），讓集團業主建立完全自定義、顆粒度細緻的操作權限。

6.2 資料加密

所有敏感資料（玩家帳號、交易紀錄、配置資訊）使用 **AES-256-GCM** 加密演算法進行加密儲存。通訊採用 HTTPS 加密協議，確保資料傳輸安全。

6.3 交易簽名驗證

所有交易使用 **RSA** 數位簽名進行驗證，確保交易的真實性與完整性，防止交易被竄改。

6.4 稽核日誌

記錄所有重要操作的詳細日誌，包括操作者、操作時間、操作內容、操作結果等資訊，支援追蹤與審計。

---

7. 效能設計

7.1 效能優化

- 使用資料庫索引優化查詢效能。
- 支援 API 請求的非同步處理，提高系統吞吐量。

---

8. 部署架構

8.1 集中式後台部署

集中式後台部署在雲端或資料中心。前端與後端服務皆使用容器化部署（Docker），支援自動擴展。資料庫使用主從複製實現高可用。

8.2 單機本地後台部署

單機本地後台與遊戲引擎部署在同一台機器上。支援 Windows、Linux、macOS 等作業系統。使用 Node.js 執行環境，SQLite 作為本地資料庫。

8.3 網路拓撲

- 集中式後台與單機本地後台通過 Internet 連線。
- 單機本地後台與遊戲引擎通過本地網路連線。
- 機台端應用程式與遊戲引擎通過本地網路連線。

---

9. 附錄

9.1 技術總結

層次	技術	說明
前端	Blazor Server + Radzen + Bootstrap 5	統一的前端技術
後端	ASP.NET Core 8	.NET 8 後端框架
集中式資料庫	PostgreSQL 15+	關聯式資料庫
單機本地資料庫	SQLite	本地資料儲存
通訊	HTTP/HTTPS + SignalR	實時通訊
加密	AES-256-GCM + RSA	資料安全